Gold Price Prediction Using Economic Indicators

Parth Yashwant Gadekar

1. Data Import & Preparation

```r
# Load dataset (make sure Gold_Macro_Data.csv is in the same directory)
gold <- read_csv("Gold_Macro_Data.csv")
```

```
## Rows: 1718 Columns: 81
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl  (80): Open, High, Low, Close, Adj Close, Volume, SP_open, SP_high, SP_l...
## date  (1): Date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Preview dataset structure and variable types
glimpse(gold)
```

```
## Rows: 1,718
## Columns: 81
## $ Date            <date> 2011-12-15, 2011-12-16, 2011-12-19, 2011-12-20, 2011-~
## $ Open            <dbl> 154.74, 154.31, 155.48, 156.82, 156.98, 156.35, 156.35~
## $ High            <dbl> 154.95, 155.37, 155.86, 157.43, 157.53, 156.80, 156.49~
## $ Low             <dbl> 151.71, 153.90, 154.36, 156.58, 156.13, 155.33, 155.82~
## $ Close           <dbl> 152.33, 155.23, 154.87, 156.98, 157.16, 156.04, 156.31~
## $ `Adj Close`     <dbl> 152.33, 155.23, 154.87, 156.98, 157.16, 156.04, 156.31~
## $ Volume          <dbl> 21521900, 18124300, 12547200, 9136300, 11996100, 98884~
## $ SP_open         <dbl> 123.03, 122.23, 122.06, 122.18, 123.93, 124.63, 125.67~
## $ SP_high         <dbl> 123.20, 122.95, 122.32, 124.14, 124.36, 125.40, 126.43~
## $ SP_low          <dbl> 121.99, 121.30, 120.03, 120.37, 122.75, 124.23, 125.41~
## $ SP_close        <dbl> 122.18, 121.59, 120.29, 123.93, 124.17, 125.27, 126.39~
## $ SP_Ajclose      <dbl> 105.4412, 105.5975, 104.4685, 107.6298, 107.8382, 108.~
## $ SP_volume       <dbl> 199109200, 220481400, 183903000, 225418100, 194230900,~
## $ DJ_open         <dbl> 11825.29, 11870.25, 11866.54, 11769.21, 12103.58, 1210~
## $ DJ_high         <dbl> 11967.84, 11968.18, 11925.88, 12117.13, 12119.70, 1218~
## $ DJ_low          <dbl> 11825.22, 11819.31, 11735.19, 11768.83, 11999.44, 1210~
## $ DJ_close        <dbl> 11868.81, 11866.39, 11766.26, 12103.58, 12107.74, 1216~
## $ DJ_Ajclose      <dbl> 11868.81, 11866.39, 11766.26, 12103.58, 12107.74, 1216~
## $ DJ_volume       <dbl> 136930000, 389520000, 135170000, 165180000, 163250000,~
## $ EG_open         <dbl> 74.55, 73.60, 69.10, 66.45, 67.10, 68.40, 70.05, 70.80~
## $ EG_high         <dbl> 76.15, 75.10, 69.80, 68.10, 69.40, 70.00, 71.75, 71.15~
## $ EG_low          <dbl> 72.15, 73.35, 64.20, 66.00, 66.90, 66.70, 69.90, 67.55~
## $ EG_close        <dbl> 72.90, 74.90, 64.70, 67.00, 68.50, 69.75, 71.60, 67.65~
```

```
## $ EG_Ajclose      <dbl> 70.43176, 72.36404, 62.50938, 64.73151, 66.18072, 67.3~
## $ EG_volume       <dbl> 787900, 896600, 2096700, 875300, 837600, 780600, 62120~
## $ EU_Price        <dbl> 1.3018, 1.3035, 1.2995, 1.3079, 1.3045, 1.3051, 1.3043~
## $ EU_open         <dbl> 1.2982, 1.3020, 1.3043, 1.3003, 1.3079, 1.3047, 1.3051~
## $ EU_high         <dbl> 1.3051, 1.3087, 1.3044, 1.3133, 1.3197, 1.3121, 1.3098~
## $ EU_low          <dbl> 1.2957, 1.2997, 1.2981, 1.2994, 1.3024, 1.3018, 1.3026~
## $ EU_Trend        <dbl> 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, ~
## $ OF_Price        <dbl> 105.09, 103.35, 103.64, 106.73, 107.71, 107.89, 107.96~
## $ OF_Open         <dbl> 104.88, 103.51, 103.63, 104.30, 107.15, 107.92, 107.77~
## $ OF_High         <dbl> 106.50, 104.56, 104.57, 107.27, 108.17, 108.50, 108.55~
## $ OF_Low          <dbl> 104.88, 102.46, 102.37, 103.91, 106.16, 107.29, 107.40~
## $ OF_Volume       <dbl> 14330, 140080, 147880, 170240, 145090, 111160, 64610, ~
## $ OF_Trend        <dbl> 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, ~
## $ OS_Price        <dbl> 93.42, 93.79, 94.09, 95.55, 99.01, 99.47, 99.85, 101.2~
## $ OS_Open         <dbl> 94.91, 93.43, 93.77, 96.39, 97.54, 99.04, 99.49, 99.94~
## $ OS_High         <dbl> 96.00, 94.80, 94.43, 99.70, 99.26, 100.06, 100.24, 101~
## $ OS_Low          <dbl> 93.33, 92.53, 92.55, 96.39, 96.81, 98.52, 99.25, 99.39~
## $ OS_Trend        <dbl> 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, ~
## $ SF_Price        <dbl> 53604, 53458, 52961, 53487, 53148, 52894, 53067, 52711~
## $ SF_Open         <dbl> 54248, 53650, 53400, 52795, 53519, 53000, 53000, 53014~
## $ SF_High         <dbl> 54248, 54030, 53400, 53575, 54184, 53460, 53348, 53099~
## $ SF_Low          <dbl> 52316, 52890, 52544, 52595, 52937, 52690, 52960, 52588~
## $ SF_Volume       <dbl> 119440, 65390, 67280, 55130, 75950, 56530, 27510, 2745~
## $ SF_Trend        <dbl> 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, ~
## $ USB_Price       <dbl> 1.911, 1.851, 1.810, 1.927, 1.970, 1.953, 2.021, 2.002~
## $ USB_Open        <dbl> 1.911, 1.851, 1.810, 1.927, 1.970, 1.953, 2.021, 2.002~
## $ USB_High        <dbl> 1.911, 1.851, 1.810, 1.927, 1.970, 1.953, 2.021, 2.002~
## $ USB_Low         <dbl> 1.911, 1.851, 1.810, 1.927, 1.970, 1.953, 2.021, 2.002~
## $ USB_Trend       <dbl> 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, ~
## $ PLT_Price       <dbl> 1414.65, 1420.25, 1411.10, 1434.75, 1429.05, 1425.90, ~
## $ PLT_Open        <dbl> 1420.30, 1414.75, 1422.65, 1408.95, 1434.40, 1429.00, ~
## $ PLT_High        <dbl> 1423.35, 1431.75, 1427.60, 1436.55, 1453.75, 1435.25, ~
## $ PLT_Low         <dbl> 1376.85, 1400.70, 1404.60, 1408.15, 1417.65, 1415.90, ~
## $ PLT_Trend       <dbl> 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, ~
## $ PLD_Price       <dbl> 618.85, 623.65, 608.80, 626.65, 635.90, 652.45, 665.55~
## $ PLD_Open        <dbl> 614.70, 622.60, 626.00, 622.45, 625.70, 632.00, 664.95~
## $ PLD_High        <dbl> 615.00, 623.45, 630.00, 622.45, 641.50, 650.80, 665.05~
## $ PLD_Low         <dbl> 614.60, 622.30, 608.60, 622.45, 623.80, 632.00, 664.95~
## $ PLD_Trend       <dbl> 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, ~
## $ RHO_PRICE       <dbl> 1425, 1400, 1400, 1400, 1400, 1400, 1400, 1400, 1400, ~
## $ USDI_Price      <dbl> 80.341, 80.249, 80.207, 80.273, 80.350, 80.306, 80.236~
## $ USDI_Open       <dbl> 80.565, 80.175, 80.300, 80.890, 80.105, 80.360, 80.210~
## $ USDI_High       <dbl> 80.630, 80.395, 80.470, 80.940, 80.445, 80.505, 80.375~
## $ USDI_Low        <dbl> 80.130, 79.935, 80.125, 80.035, 79.550, 79.955, 80.060~
## $ USDI_Volume     <dbl> 22850, 13150, 970, 22950, 24140, 13240, 7170, 5660, 21~
## $ USDI_Trend      <dbl> 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, ~
## $ GDX_Open        <dbl> 53.01, 52.50, 52.49, 52.38, 53.15, 52.56, 52.59, 52.37~
## $ GDX_High        <dbl> 53.14, 53.18, 52.55, 53.25, 53.43, 52.62, 52.79, 52.52~
## $ GDX_Low         <dbl> 51.57, 52.04, 51.03, 52.37, 52.42, 51.77, 52.35, 51.68~
## $ GDX_Close       <dbl> 51.68, 52.68, 51.17, 52.99, 52.96, 52.40, 52.79, 51.87~
## $ `GDX_Adj Close` <dbl> 48.97388, 49.92151, 48.49058, 50.21528, 50.18685, 49.6~
## $ GDX_Volume      <dbl> 20605600, 16285400, 15120200, 11644900, 8724300, 13179~
## $ USO_Open        <dbl> 36.90, 36.18, 36.39, 37.30, 37.67, 38.27, 38.47, 38.49~
## $ USO_High        <dbl> 36.94, 36.50, 36.45, 37.61, 38.24, 38.55, 38.61, 39.21~
```

```
## $ USO_Low         <dbl> 36.05, 35.73, 35.93, 37.22, 37.52, 38.21, 38.37, 38.46~
## $ USO_Close       <dbl> 36.13, 36.27, 36.20, 37.56, 38.11, 38.28, 38.49, 39.06~
## $ `USO_Adj Close` <dbl> 36.13, 36.27, 36.20, 37.56, 38.11, 38.28, 38.49, 39.06~
## $ USO_Volume      <dbl> 12616700, 12578800, 7418200, 10041600, 10728000, 53768~
```

```r
# Convert Date column to Date format
gold <- gold %>%
  mutate(Date = as.Date(Date))

# Handle missing values via forward fill (fills missing data downwards and upwards)
gold <- gold %>%
  arrange(Date) %>%
  fill(everything(), .direction = "downup")

# Create lagged and Simple Moving Average (SMA) features
gold <- gold %>%
  mutate(
    SMA_30 = zoo::rollmean(Close, 30, fill = NA, align = "right"),
    Return_1d = Close / lag(Close) - 1,
    Trend = factor(ifelse(lead(Close) > Close, "Up", "Down"))
  ) %>%
  drop_na()
```

2. Exploratory Data Analysis

```r
# Summary statistics for all numerical variables (excluding Date)
gold %>%
  select(-Date) %>%
  summary()
```

```
##       Open            High            Low            Close
##  Min.   :100.9   Min.   :101.0   Min.   :100.2   Min.   :100.5
##  1st Qu.:116.1   1st Qu.:116.5   1st Qu.:115.6   1st Qu.:116.0
##  Median :121.7   Median :122.1   Median :121.1   Median :121.6
##  Mean   :126.8   Mean   :127.3   Mean   :126.3   Mean   :126.8
##  3rd Qu.:128.0   3rd Qu.:128.5   3rd Qu.:127.5   3rd Qu.:128.0
##  Max.   :173.2   Max.   :174.1   Max.   :172.9   Max.   :173.6
##    Adj Close         Volume            SP_open         SP_high
##  Min.   :100.5   Min.   : 1501600   Min.   :127.8   Min.   :128.7
##  1st Qu.:116.0   1st Qu.: 5377175   1st Qu.:176.0   1st Qu.:176.9
##  Median :121.6   Median : 7419150   Median :206.0   Median :207.0
##  Mean   :126.8   Mean   : 8394801   Mean   :205.8   Mean   :206.7
##  3rd Qu.:128.0   3rd Qu.:10130425   3rd Qu.:237.7   3rd Qu.:238.7
##  Max.   :173.6   Max.   :93804200   Max.   :293.1   Max.   :293.9
##      SP_low         SP_close       SP_Ajclose      SP_volume
##  Min.   :127.1   Min.   :128.1   Min.   :111.7   Min.   : 27856500
##  1st Qu.:175.2   1st Qu.:176.2   1st Qu.:158.8   1st Qu.: 73631725
##  Median :204.8   Median :205.9   Median :192.4   Median : 99102700
##  Mean   :204.8   Mean   :205.8   Mean   :193.6   Mean   :109179974
##  3rd Qu.:237.0   3rd Qu.:237.7   3rd Qu.:229.3   3rd Qu.:134471825
##  Max.   :291.8   Max.   :293.6   Max.   :290.6   Max.   :507244300
```

```
##     DJ_open         DJ_high          DJ_low         DJ_close
##  Min.   :12101   Min.   :12144   Min.   :12035   Min.   :12101
##  1st Qu.:15628   1st Qu.:15677   1st Qu.:15541   1st Qu.:15628
##  Median :17644   Median :17740   Median :17549   Median :17647
##  Mean   :18258   Mean   :18341   Mean   :18170   Mean   :18261
##  3rd Qu.:20916   3rd Qu.:20957   3rd Qu.:20860   3rd Qu.:20908
##  Max.   :26833   Max.   :26952   Max.   :26789   Max.   :26828
##    DJ_Ajclose      DJ_volume            EG_open         EG_high
##  Min.   :12101   Min.   : 40350000   Min.   : 2.85   Min.   : 2.85
##  1st Qu.:15628   1st Qu.: 91912500   1st Qu.:13.99   1st Qu.:14.45
##  Median :17647   Median :120070000   Median :21.70   Median :22.45
##  Mean   :18261   Mean   :178422162   Mean   :27.57   Mean   :28.10
##  3rd Qu.:20908   3rd Qu.:265525000   3rd Qu.:36.30   3rd Qu.:37.18
##  Max.   :26828   Max.   :900510000   Max.   :80.20   Max.   :81.00
##      EG_low         EG_close       EG_Ajclose      EG_volume
##  Min.   : 2.75   Min.   : 2.80   Min.   : 2.80   Min.   :  164500
##  1st Qu.:13.64   1st Qu.:14.05   1st Qu.:14.03   1st Qu.:  705750
##  Median :21.05   Median :21.57   Median :21.46   Median :  974450
##  Mean   :26.96   Mean   :27.50   Mean   :27.11   Mean   : 1143253
##  3rd Qu.:35.60   3rd Qu.:36.31   3rd Qu.:35.90   3rd Qu.: 1351150
##  Max.   :77.90   Max.   :79.80   Max.   :78.00   Max.   :10061200
##     EU_Price        EU_open         EU_high          EU_low
##  Min.   :1.039   Min.   :1.039   Min.   :1.042   Min.   :1.034
##  1st Qu.:1.120   1st Qu.:1.120   1st Qu.:1.125   1st Qu.:1.116
##  Median :1.181   Median :1.181   Median :1.185   Median :1.177
##  Mean   :1.207   Mean   :1.207   Mean   :1.212   Mean   :1.202
##  3rd Qu.:1.306   3rd Qu.:1.306   3rd Qu.:1.310   3rd Qu.:1.300
##  Max.   :1.393   Max.   :1.393   Max.   :1.399   Max.   :1.391
##     EU_Trend         OF_Price         OF_Open          OF_High
##  Min.   :0.0000   Min.   : 27.88   Min.   : 27.99   Min.   : 28.75
##  1st Qu.:0.0000   1st Qu.: 51.99   1st Qu.: 52.01   1st Qu.: 52.73
##  Median :0.0000   Median : 69.11   Median : 69.14   Median : 69.76
##  Mean   :0.4935   Mean   : 76.96   Mean   : 76.98   Mean   : 77.84
##  3rd Qu.:1.0000   3rd Qu.:107.45   3rd Qu.:107.45   3rd Qu.:108.25
##  Max.   :1.0000   Max.   :126.22   Max.   :126.16   Max.   :128.40
##      OF_Low         OF_Volume         OF_Trend         OS_Price
##  Min.   : 27.10   Min.   : 11520   Min.   :0.0000   Min.   : 26.55
##  1st Qu.: 51.31   1st Qu.:178145   1st Qu.:0.0000   1st Qu.: 48.84
##  Median : 68.39   Median :224390   Median :0.0000   Median : 63.63
##  Mean   : 76.06   Mean   :227263   Mean   :0.4976   Mean   : 69.66
##  3rd Qu.:106.78   3rd Qu.:283368   3rd Qu.:1.0000   3rd Qu.: 93.87
##  Max.   :125.00   Max.   :567760   Max.   :1.0000   Max.   :110.30
##     OS_Open          OS_High           OS_Low          OS_Trend
##  Min.   : 27.34   Min.   : 27.61   Min.   : 26.18   Min.   :0.000
##  1st Qu.: 48.90   1st Qu.: 49.65   1st Qu.: 48.08   1st Qu.:0.000
##  Median : 63.66   Median : 64.33   Median : 63.08   Median :1.000
##  Mean   : 69.79   Mean   : 70.63   Mean   : 68.85   Mean   :0.503
##  3rd Qu.: 93.99   3rd Qu.: 94.74   3rd Qu.: 93.11   3rd Qu.:1.000
##  Max.   :110.34   Max.   :112.28   Max.   :109.15   Max.   :1.000
##     SF_Price        SF_Open         SF_High          SF_Low
##  Min.   :33170   Min.   :33146   Min.   :33566   Min.   :32626
##  1st Qu.:37982   1st Qu.:37997   1st Qu.:38270   1st Qu.:37642
##  Median :40395   Median :40407   Median :40740   Median :40128
##  Mean   :43115   Mean   :43140   Mean   :43499   Mean   :42747
```

```
##  3rd Qu.:46048   3rd Qu.:46063   3rd Qu.:46449   3rd Qu.:45577
##  Max.   :65292   Max.   :65400   Max.   :65723   Max.   :64132
##    SF_Volume         SF_Trend        USB_Price        USB_Open
##  Min.   :    40   Min.   :0.0000   Min.   :1.358   Min.   :1.366
##  1st Qu.: 14178   1st Qu.:0.0000   1st Qu.:1.909   1st Qu.:1.908
##  Median : 19435   Median :0.0000   Median :2.268   Median :2.269
##  Mean   : 26127   Mean   :0.4805   Mean   :2.268   Mean   :2.268
##  3rd Qu.: 29515   3rd Qu.:1.0000   3rd Qu.:2.601   3rd Qu.:2.601
##  Max.   :203730   Max.   :1.0000   Max.   :3.239   Max.   :3.237
##     USB_High        USB_Low        USB_Trend        PLT_Price
##  Min.   :1.391   Min.   :1.321   Min.   :0.0000   Min.   : 775.6
##  1st Qu.:1.925   1st Qu.:1.885   1st Qu.:0.0000   1st Qu.: 943.3
##  Median :2.297   Median :2.239   Median :0.0000   Median :1086.1
##  Mean   :2.292   Mean   :2.244   Mean   :0.4905   Mean   :1179.2
##  3rd Qu.:2.625   3rd Qu.:2.579   3rd Qu.:1.0000   3rd Qu.:1440.3
##  Max.   :3.261   Max.   :3.231   Max.   :1.0000   Max.   :1737.6
##     PLT_Open         PLT_High         PLT_Low         PLT_Trend
##  Min.   : 765.3   Min.   : 786.5   Min.   : 756.0   Min.   :0.000
##  1st Qu.: 942.7   1st Qu.: 951.7   1st Qu.: 935.3   1st Qu.:0.000
##  Median :1086.8   Median :1095.7   Median :1076.2   Median :0.000
##  Mean   :1179.8   Mean   :1189.5   Mean   :1168.8   Mean   :0.484
##  3rd Qu.:1441.8   3rd Qu.:1453.4   3rd Qu.:1431.4   3rd Qu.:1.000
##  Max.   :1737.8   Max.   :1742.9   Max.   :1717.2   Max.   :1.000
##    PLD_Price         PLD_Open         PLD_High         PLD_Low
##  Min.   : 470.4   Min.   : 458.6   Min.   : 473.1   Min.   : 458.6
##  1st Qu.: 667.8   1st Qu.: 668.5   1st Qu.: 674.1   1st Qu.: 660.9
##  Median : 750.3   Median : 750.2   Median : 756.1   Median : 745.0
##  Mean   : 768.5   Mean   : 768.1   Mean   : 775.3   Mean   : 761.2
##  3rd Qu.: 854.6   3rd Qu.: 851.1   3rd Qu.: 860.9   3rd Qu.: 844.8
##  Max.   :1196.0   Max.   :1196.0   Max.   :1208.7   Max.   :1181.5
##    PLD_Trend        RHO_PRICE      USDI_Price       USDI_Open
##  Min.   :0.0000   Min.   :   0   Min.   : 78.30   Min.   : 78.22
##  1st Qu.:0.0000   1st Qu.: 785   1st Qu.: 81.52   1st Qu.: 81.50
##  Median :1.0000   Median :1100   Median : 93.04   Median : 93.04
##  Mean   :0.5284   Mean   :1127   Mean   : 89.97   Mean   : 89.96
##  3rd Qu.:1.0000   3rd Qu.:1265   3rd Qu.: 96.13   3rd Qu.: 96.17
##  Max.   :1.0000   Max.   :2600   Max.   :103.29   Max.   :103.35
##    USDI_High         USDI_Low       USDI_Volume       USDI_Trend
##  Min.   : 78.64   Min.   : 78.12   Min.   :    60   Min.   :0.0000
##  1st Qu.: 81.74   1st Qu.: 81.31   1st Qu.: 18218   1st Qu.:0.0000
##  Median : 93.36   Median : 92.81   Median : 24565   Median :1.0000
##  Mean   : 90.25   Mean   : 89.67   Mean   : 27734   Mean   :0.5154
##  3rd Qu.: 96.51   3rd Qu.: 95.81   3rd Qu.: 33883   3rd Qu.:1.0000
##  Max.   :103.81   Max.   :102.97   Max.   :142820   Max.   :1.0000
##     GDX_Open        GDX_High        GDX_Low        GDX_Close
##  Min.   :12.70   Min.   :12.92   Min.   :12.40   Min.   :12.47
##  1st Qu.:20.59   1st Qu.:20.88   1st Qu.:20.28   1st Qu.:20.54
##  Median :23.06   Median :23.27   Median :22.79   Median :22.98
##  Mean   :26.30   Mean   :26.62   Mean   :25.94   Mean   :26.26
##  3rd Qu.:26.92   3rd Qu.:27.30   3rd Qu.:26.55   3rd Qu.:26.93
##  Max.   :57.52   Max.   :57.94   Max.   :56.77   Max.   :57.47
##  GDX_Adj Close     GDX_Volume           USO_Open        USO_High
##  Min.   :12.27   Min.   :  5593600   Min.   : 7.82   Min.   : 8.03
##  1st Qu.:20.06   1st Qu.: 23590550   1st Qu.:11.40   1st Qu.:11.48
```

```
##  Median :22.63   Median : 37849100   Median :15.78   Median :16.04
##  Mean   :25.51   Mean   : 44120475   Mean   :21.84   Mean   :22.03
##  3rd Qu.:26.13   3rd Qu.: 57394825   3rd Qu.:34.20   3rd Qu.:34.43
##  Max.   :54.62   Max.   :232153600   Max.   :41.60   Max.   :42.30
##     USO_Low        USO_Close     USO_Adj Close     USO_Volume
##  Min.   : 7.67   Min.   : 7.96   Min.   : 7.96   Min.   :  1035100
##  1st Qu.:11.27   1st Qu.:11.37   1st Qu.:11.37   1st Qu.:  6208650
##  Median :15.59   Median :15.78   Median :15.78   Median : 16443550
##  Mean   :21.64   Mean   :21.84   Mean   :21.84   Mean   : 19384237
##  3rd Qu.:33.92   3rd Qu.:34.17   3rd Qu.:34.17   3rd Qu.: 26998100
##  Max.   :41.30   Max.   :42.01   Max.   :42.01   Max.   :110265700
##      SMA_30        Return_1d           Trend
##  Min.   :102.3   Min.   :-0.0878083   Down:848
##  1st Qu.:116.0   1st Qu.:-0.0050308   Up  :840
##  Median :121.7   Median : 0.0000000
##  Mean   :127.2   Mean   :-0.0001483
##  3rd Qu.:127.6   3rd Qu.: 0.0044567
##  Max.   :170.4   Max.   : 0.0746326
```

```r
# Correlation matrix focused on Close
gold %>%
  select(where(is.numeric)) %>%
  correlate() %>%
  focus(Close) %>%
  arrange(desc(abs(Close))) %>%
  kable(caption = "Correlation of Variables with Close") %>%
  kable_styling(full_width = FALSE)
```

```
## Correlation computed with
## * Method: 'pearson'
## * Missing treated using: 'pairwise.complete.obs'
```

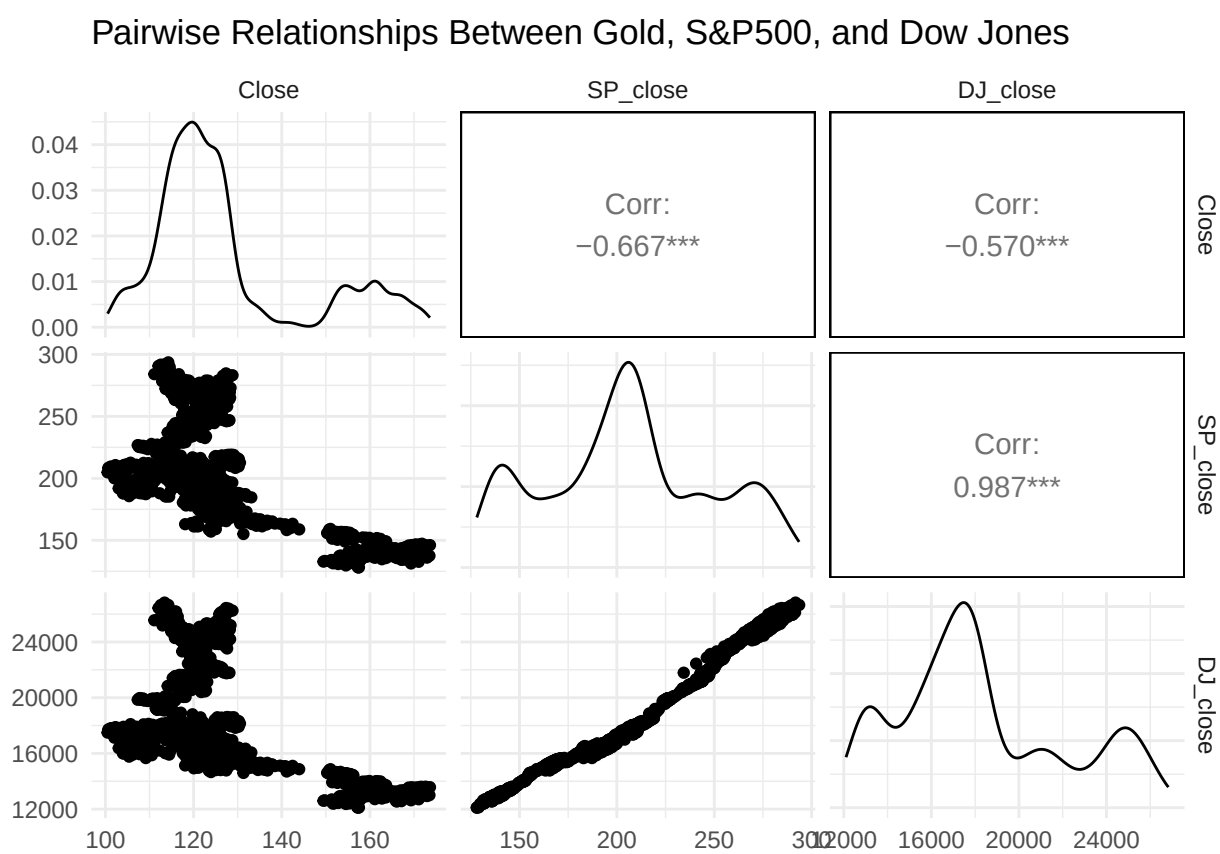
Table 1: Correlation of Variables with Close

term	Close
Adj Close	1.0000000
Low	0.9995454
High	0.9995250
Open	0.9989848
GDX_Low	0.9789935
GDX_Close	0.9788610
GDX_High	0.9786328
SMA_30	0.9784791
GDX_Adj Close	0.9782024
GDX_Open	0.9781801
SF_Low	0.9476678
SF_Price	0.9470270
SF_Open	0.9453048
SF_High	0.9447305
EG_low	0.8562557
EG_open	0.8552259

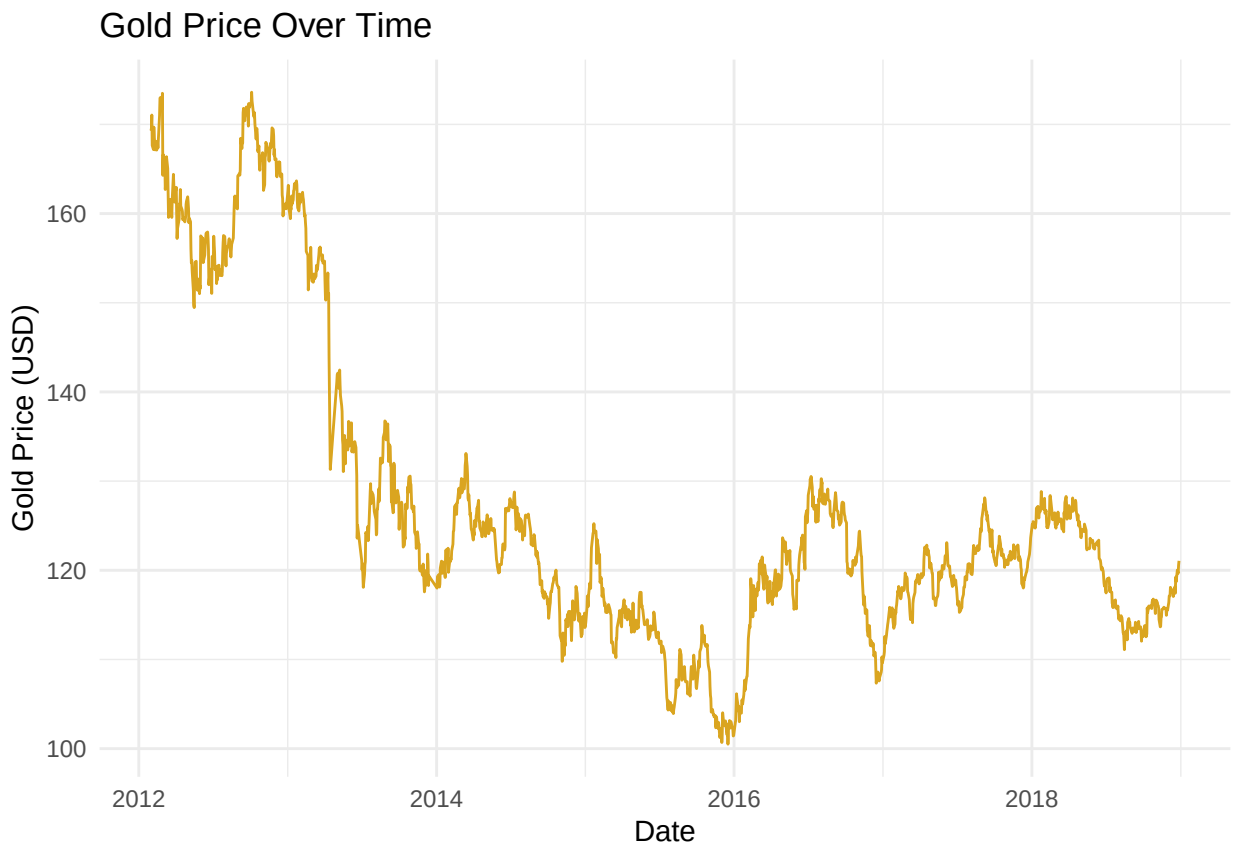
EG_close	0.8549604
EG_high	0.8536140
EG_Ajclose	0.8518333
PLT_Price	0.7720709
PLT_High	0.7717865
PLT_Low	0.7706301
PLT_Open	0.7703643
USDI_Low	-0.7132843
USDI_Price	-0.7129461
USDI_High	-0.7116043
USDI_Open	-0.7115873
OF_High	0.7030799
OF_Price	0.7025517
OF_Open	0.7008836
OF_Low	0.7000350
SF_Volume	0.6931640
SP_high	-0.6678429
SP_open	-0.6675231
SP_close	-0.6672404
SP_low	-0.6666133
SP_Ajclose	-0.6488816
USO_Close	0.6203428
USO_Adj Close	0.6203428
USO_High	0.6199054
USO_Open	0.6198405
USO_Low	0.6194069
OS_High	0.6192041
OS_Price	0.6181434
OS_Open	0.6173776
OS_Low	0.6164823
EU_high	0.5761401
EU_Price	0.5744867
EU_open	0.5727052
EU_low	0.5709441
DJ_high	-0.5702658
DJ_open	-0.5702462
DJ_close	-0.5699286
DJ_Ajclose	-0.5699286
DJ_low	-0.5695579
GDX_Volume	-0.4998281
USB_High	-0.4552903
USO_Volume	-0.4517806
USB_Price	-0.4329428
USB_Open	-0.4324099
USB_Low	-0.4094449
Volume	0.2382871
SP_volume	0.2289227
OF_Volume	-0.2103530
PLD_High	-0.2009726
PLD_Open	-0.1993661

PLD_Low	-0.1974158
PLD_Price	-0.1964535
USDI_Volume	-0.1945418
DJ_volume	-0.1521532
EG_volume	-0.1130282
RHO_PRICE	0.0841098
OS_Trend	0.0593659
OF_Trend	0.0477736
SF_Trend	0.0280308
USB_Trend	-0.0269716
PLD_Trend	0.0200862
EU_Trend	0.0169221
Return_1d	0.0168887
PLT_Trend	0.0113992
USDI_Trend	-0.0086153

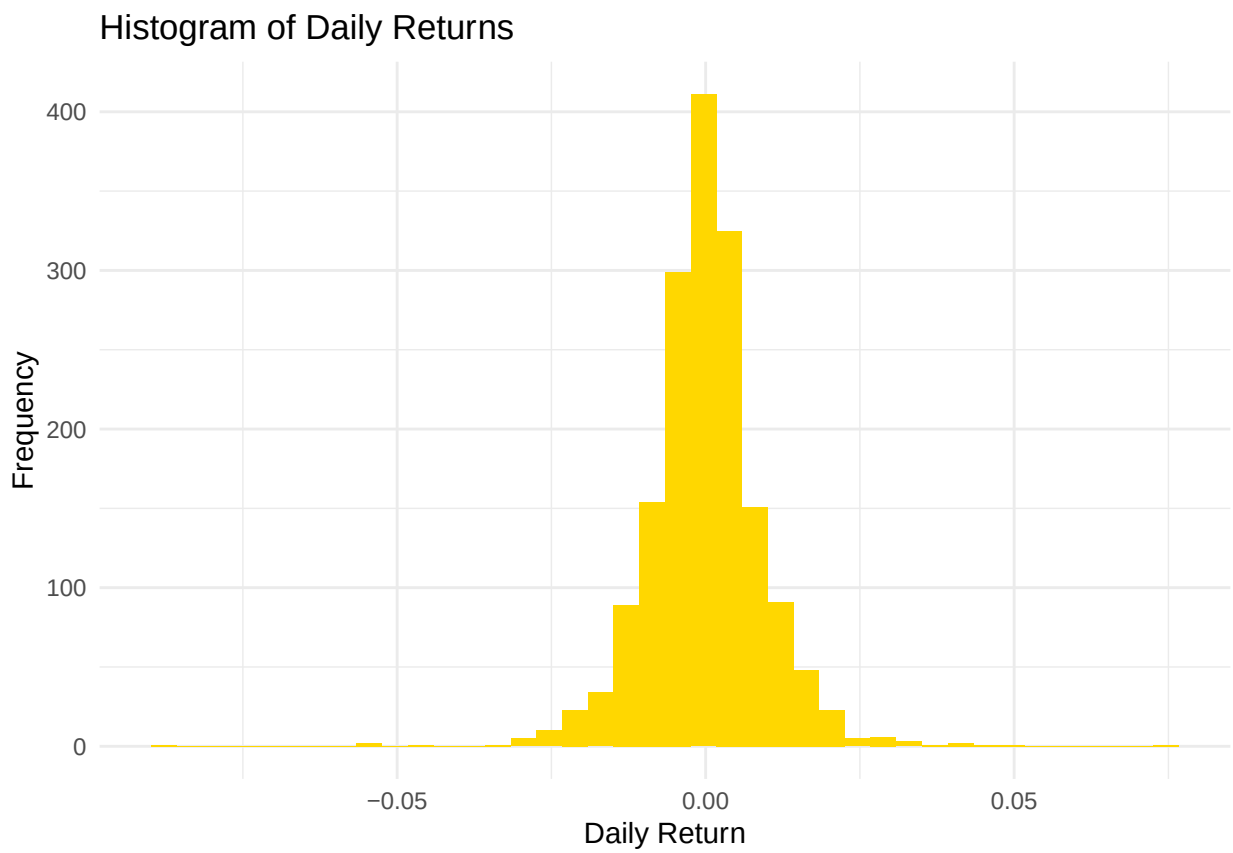
```r
# Pairwise plots with columns that exist in the dataset
ggpairs(
  gold,
  columns = c("Close", "SP_close", "DJ_close"),
  title = "Pairwise Relationships Between Gold, S&P500, and Dow Jones"
)
```



```r
# Gold price time-series plot
ggplot(gold, aes(Date, Close)) +
  geom_line(color = "goldenrod") +
  labs(title = "Gold Price Over Time", y = "Gold Price (USD)", x = "Date")
```




```r
# Histogram of daily returns
ggplot(gold, aes(Return_1d)) +
  geom_histogram(bins = 40, fill = "gold") +
  labs(title = "Histogram of Daily Returns", x = "Daily Return", y = "Frequency")
```



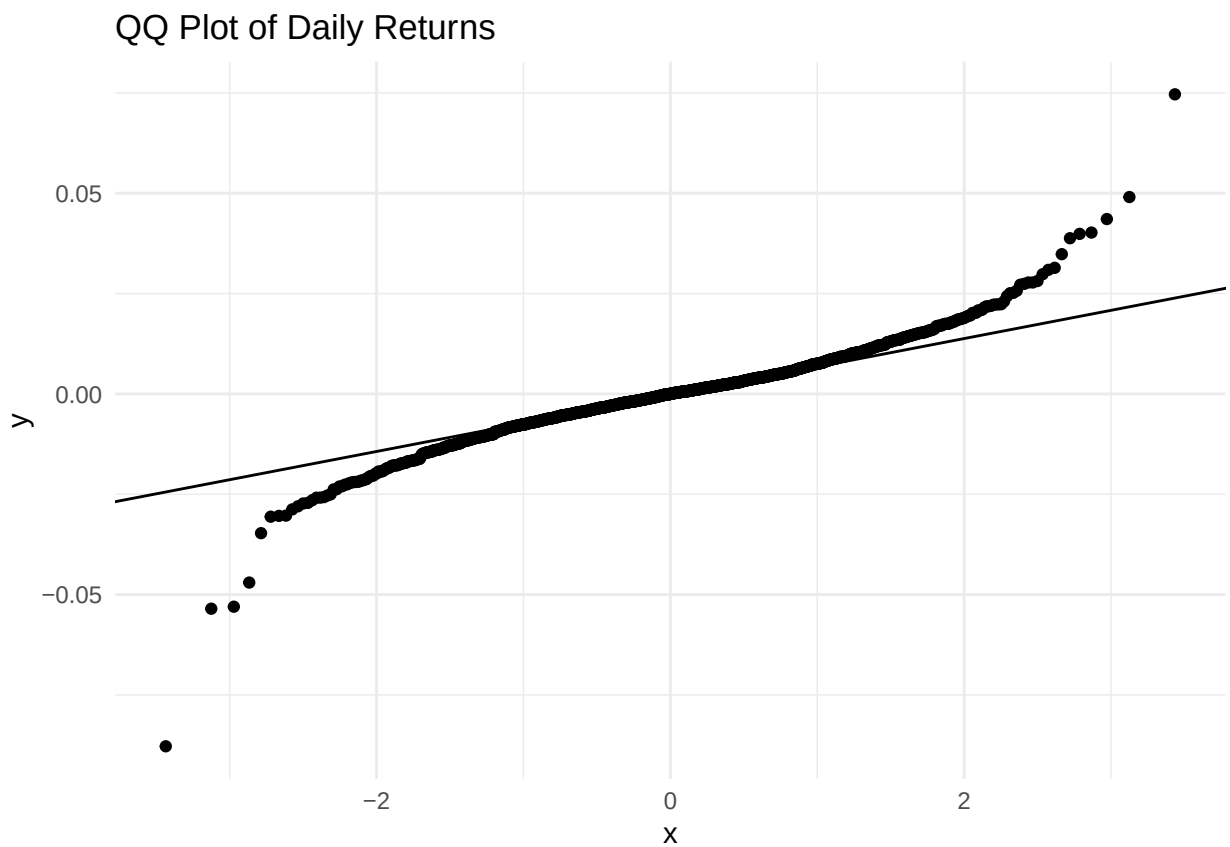

```r
# Skewness Test
library(moments)
```

```
## Warning: package 'moments' was built under R version 4.5.2
```

```r
skewness(gold$Return_1d, na.rm = TRUE)
```

```
## [1] -0.1988871
```

```r
# QQ plot for Normality
ggplot(data.frame(Return_1d = na.omit(gold$Return_1d)), aes(sample = Return_1d)) +
  stat_qq() + stat_qq_line() +
  labs(title = "QQ Plot of Daily Returns")
```

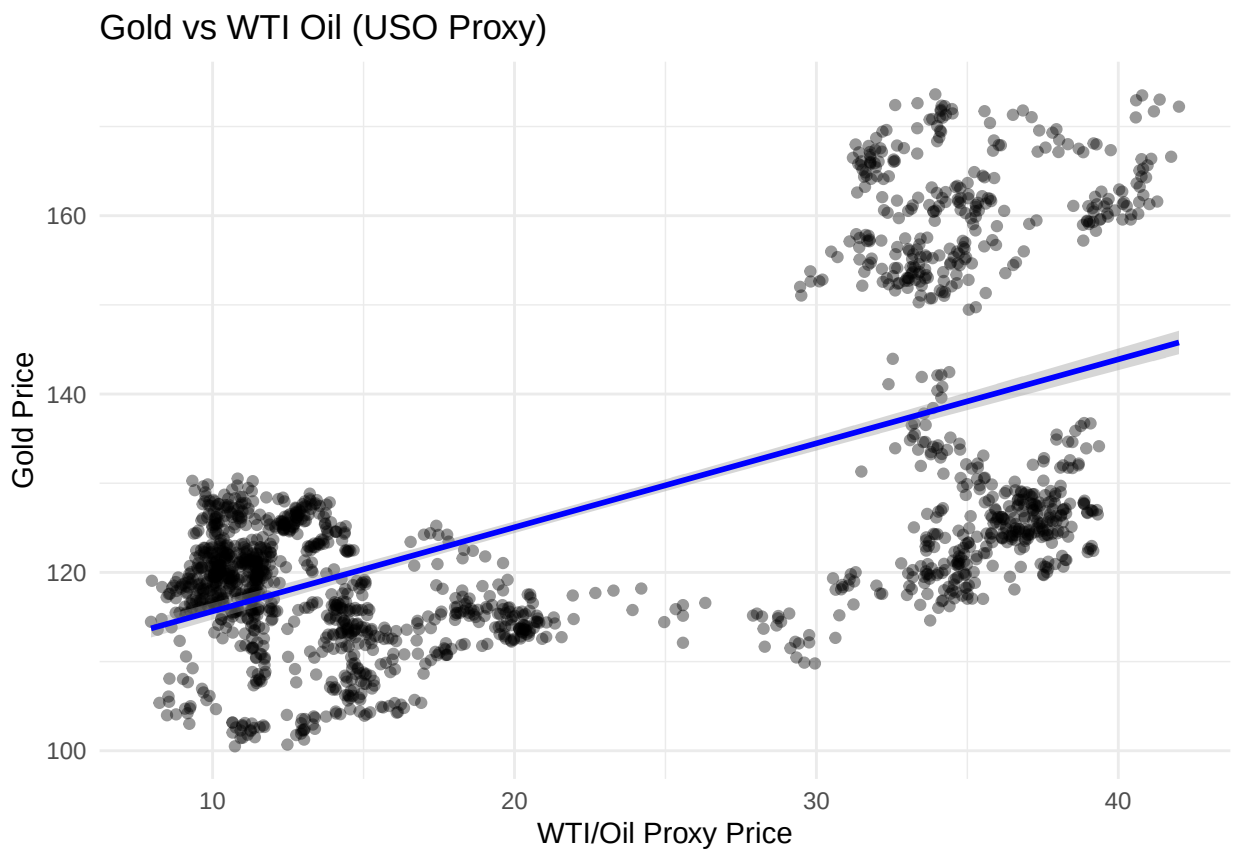



```r
# Shapiro-Wilk Normality test
shapiro.test(na.omit(gold$Return_1d))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  na.omit(gold$Return_1d)
## W = 0.93061, p-value < 2.2e-16
```

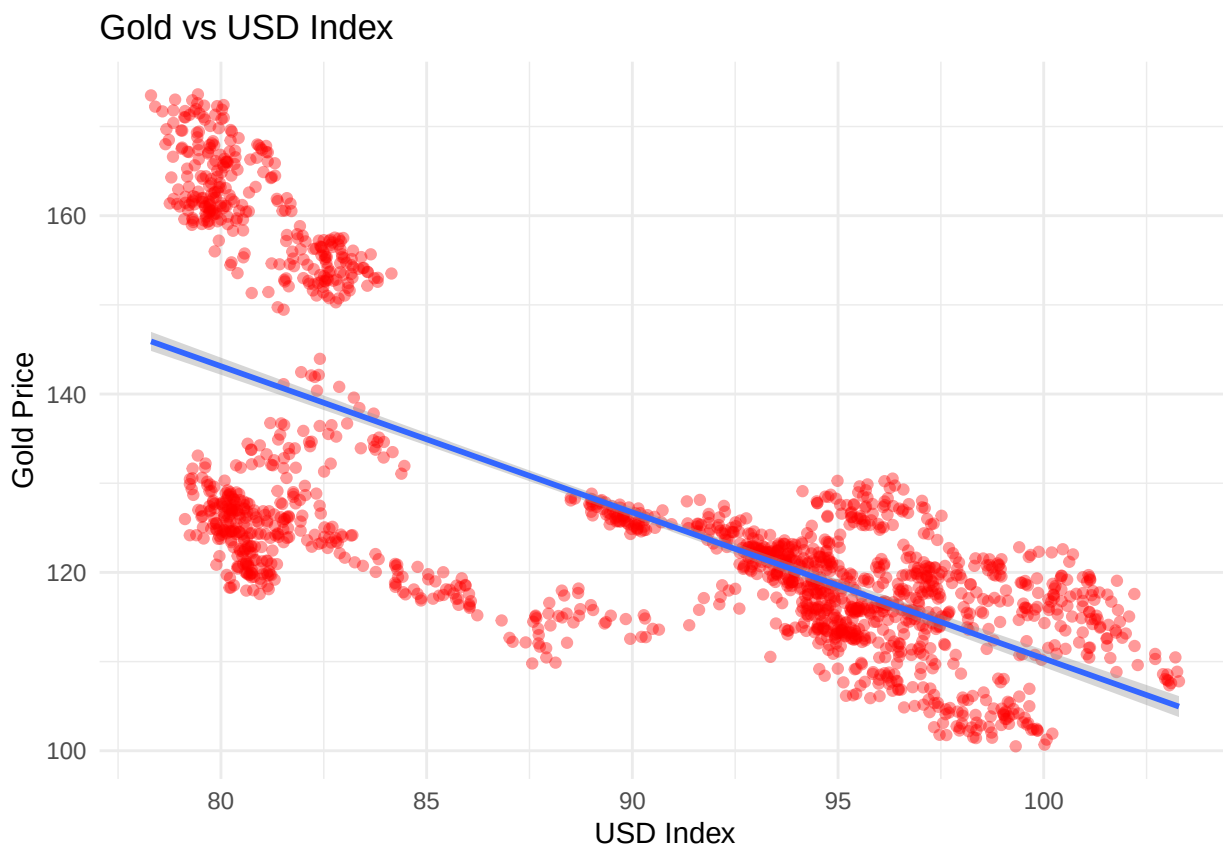
```r
# Scatterplots for Key Macro Relationships

## Gold vs WTI Oil
ggplot(gold, aes(USO_Close, Close)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", color = "blue") +
  labs(title = "Gold vs WTI Oil (USO Proxy)", x = "WTI/Oil Proxy Price", y = "Gold Price")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```r
## Gold vs USD Index
ggplot(gold, aes(USDI_Price, Close)) +
  geom_point(alpha = 0.4, color = "red") +
  geom_smooth(method = "lm") +
  labs(title = "Gold vs USD Index", x = "USD Index", y = "Gold Price")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



3. Train / Test Split

```
# Split the dataset into training (80%) and testing (20%) sets
# Using time-based splitting to preserve chronological order
gold_split <- initial_time_split(gold, prop = 0.8)

# Extract training data
train_data <- training(gold_split)

# Extract testing data
test_data  <- testing(gold_split)

# Display dimensions of each set
cat("Training set rows:", nrow(train_data), "\n")
```

```
## Training set rows: 1350
```

```
cat("Testing set rows:", nrow(test_data), "\n")
```

```
## Testing set rows: 338
```

```r
# Optional: Quick date range check for confirmation
range(train_data$Date)
```

```
## [1] "2012-01-31" "2017-08-18"
```

```r
range(test_data$Date)
```

```
## [1] "2017-08-21" "2018-12-28"
```

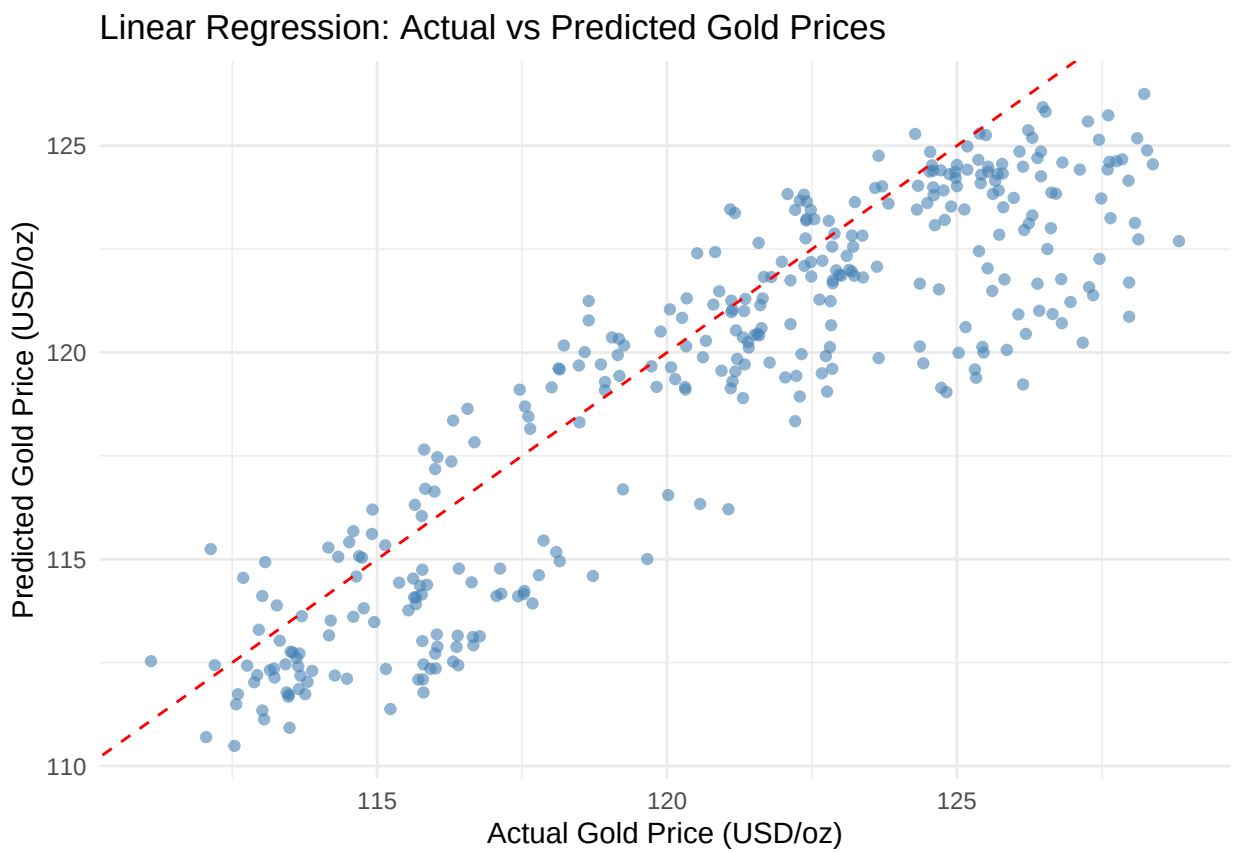
4(a). Linear Regression Model

```r
# Define the model specification using tidymodels
lin_spec <- linear_reg() %>%
  set_engine("lm")

# Create a preprocessing recipe: normalize predictors for consistent scaling
lin_recipe <- recipe(
  Close ~ SP_close + DJ_close + SMA_30 + Return_1d,
  data = train_data
) %>%
  step_normalize(all_predictors())

# Combine recipe and model into a workflow
lin_wf <- workflow() %>%
  add_model(lin_spec) %>%
  add_recipe(lin_recipe)

# Fit the model on the training data
lin_fit <- fit(lin_wf, data = train_data)

# Predict on the test data
lin_results <- predict(lin_fit, test_data) %>%
  bind_cols(test_data %>% select(Close))

# Evaluate model performance using standard regression metrics
lin_metrics <- metrics(lin_results, truth = Close, estimate = .pred)
lin_metrics %>%
  kable(caption = "Linear Regression Performance Metrics") %>%
  kable_styling(full_width = FALSE)
```

Table 2: Linear Regression Performance Metrics

.metric	.estimator	.estimate
rmse	standard	2.5048412
rsq	standard	0.8114537
mae	standard	1.9517062

```r
# Visualize actual vs predicted prices
ggplot(lin_results, aes(x = Close, y = .pred)) +
  geom_point(alpha = 0.6, color = "steelblue") +
  geom_abline(linetype = "dashed", color = "red") +
  labs(
    title = "Linear Regression: Actual vs Predicted Gold Prices",
    x = "Actual Gold Price (USD/oz)",
    y = "Predicted Gold Price (USD/oz)"
  )
```



4(b). Random Forest Regression Model

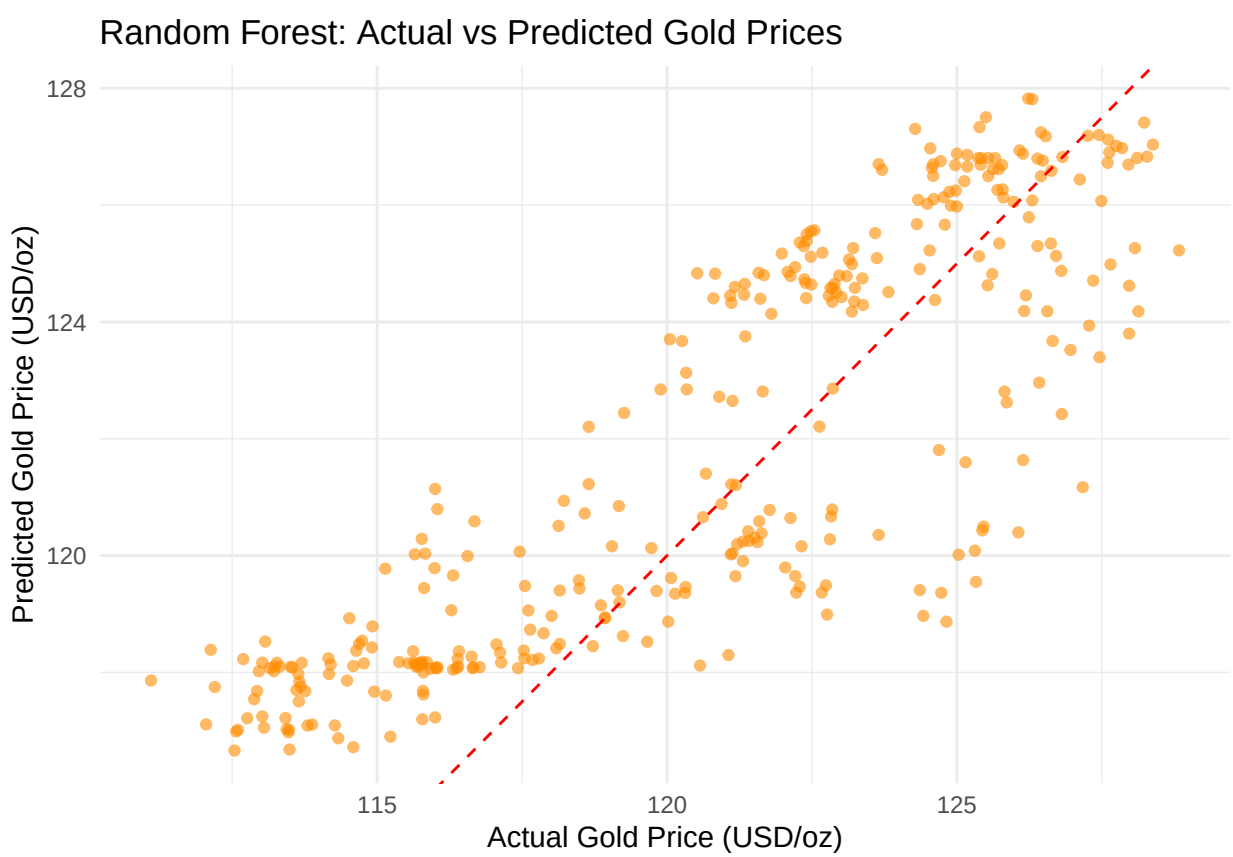
```r
rf_spec <- rand_forest(mtry = 4, trees = 300, min_n = 5) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("regression")

rf_wf <- workflow() %>%
  add_model(rf_spec) %>%
  add_recipe(lin_recipe)

rf_fit <- fit(rf_wf, data = train_data)

rf_results <- predict(rf_fit, test_data) %>%
```

```r
  bind_cols(test_data %>% select(Close))

rf_metrics <- metrics(rf_results, truth = Close, estimate = .pred)
rf_metrics %>%
  kable(caption = "Random Forest Regression Performance Metrics") %>%
  kable_styling(full_width = FALSE)
```

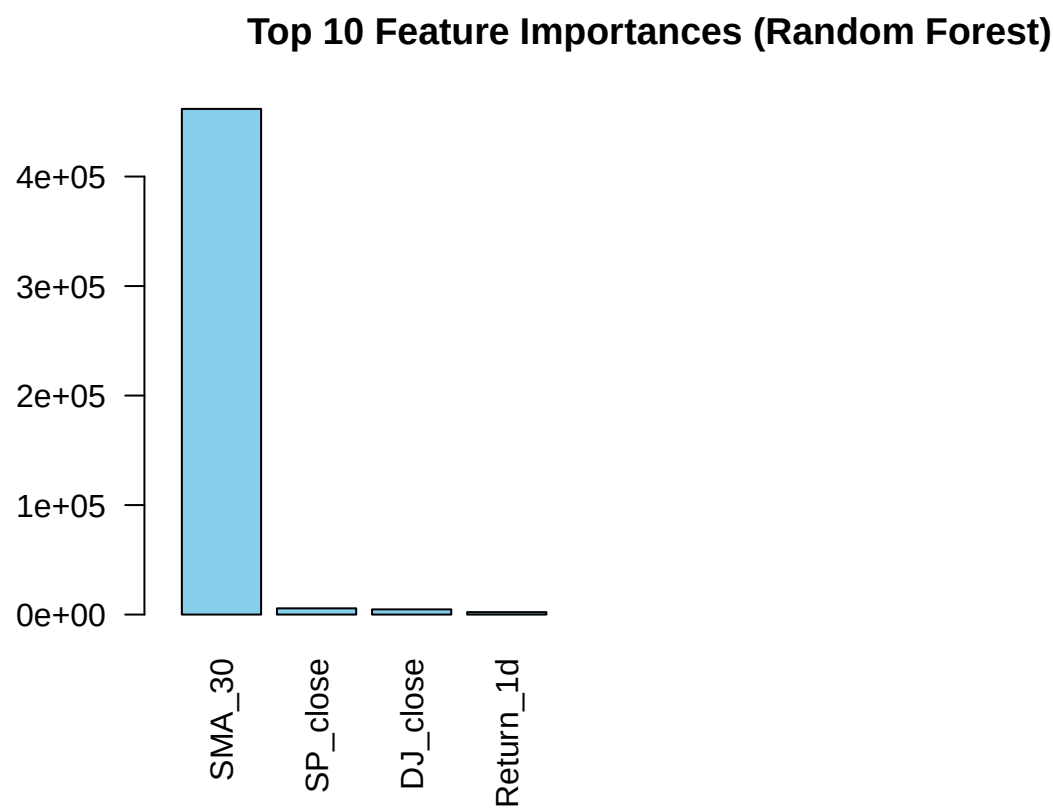
Table 3: Random Forest Regression Performance Metrics

.metric	.estimator	.estimate
rmse	standard	2.7569776
rsq	standard	0.7151428
mae	standard	2.3250416

```r
ggplot(rf_results, aes(x = Close, y = .pred)) +
  geom_point(alpha = 0.6, color = "darkorange") +
  geom_abline(linetype = "dashed", color = "red") +
  labs(
    title = "Random Forest: Actual vs Predicted Gold Prices",
    x = "Actual Gold Price (USD/oz)",
    y = "Predicted Gold Price (USD/oz)"
  )
```



```r
# Optional: Top 10 feature importances
rf_model <- extract_fit_parsnip(rf_fit) %>% pluck("fit")
rf_imp <- ranger::importance(rf_model)

barplot(
  sort(rf_imp, decreasing = TRUE)[1:10],
  main = "Top 10 Feature Importances (Random Forest)",
  las = 2,
  col = "skyblue"
)
```



5(a). K-Nearest Neighbors Classification

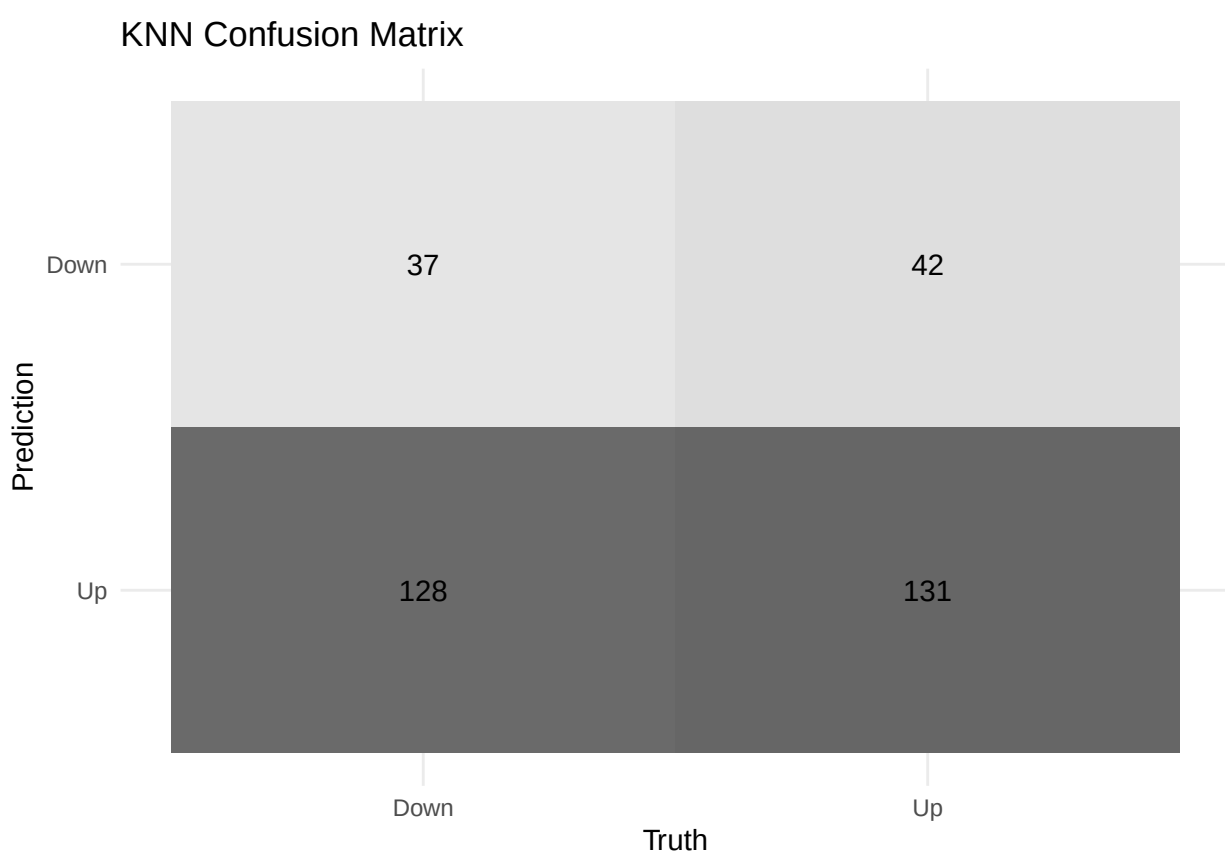
```r
class_recipe <- recipe(
  Trend ~ SP_close + DJ_close + SMA_30 + Return_1d,
  data = train_data
) %>%
  step_normalize(all_predictors())

knn_spec <- nearest_neighbor(neighbors = 7, weight_func = "rectangular") %>%
  set_engine("kknn") %>%
  set_mode("classification")
```

```r
knn_wf <- workflow() %>%
  add_model(knn_spec) %>%
  add_recipe(class_recipe)

knn_fit <- fit(knn_wf, data = train_data)

knn_preds <- predict(knn_fit, test_data) %>%
  bind_cols(test_data %>% select(Trend))

knn_metrics <- metrics(knn_preds, truth = Trend, estimate = .pred_class)
knn_metrics %>%
  kable(caption = "KNN Classification Performance Metrics") %>%
  kable_styling(full_width = FALSE)
```

Table 4: KNN Classification Performance Metrics

.metric	.estimator	.estimate
accuracy	binary	0.4970414
kap	binary	-0.0187582

```r
conf_mat(knn_preds, truth = Trend, estimate = .pred_class) %>%
  autoplot(type = "heatmap") +
  labs(title = "KNN Confusion Matrix")
```



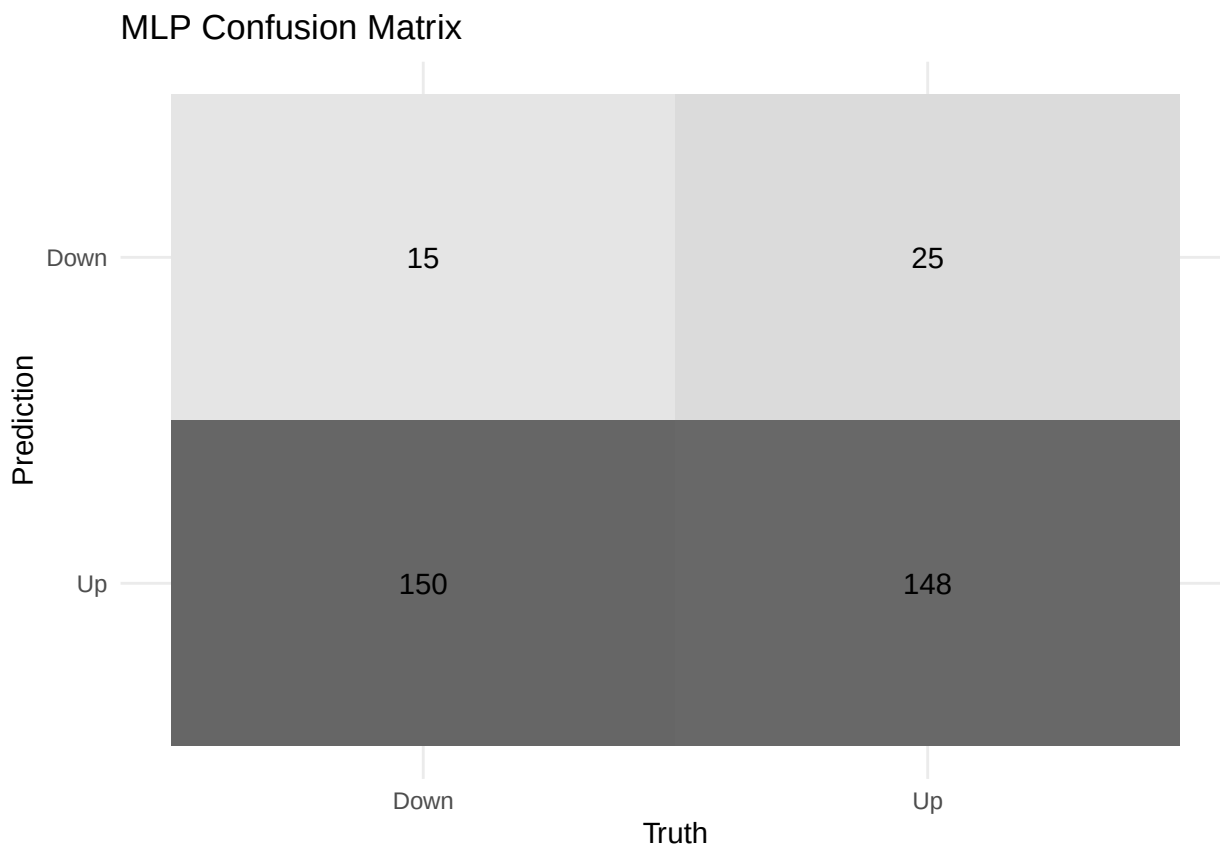
5(b). Multi-Layer Perceptron (Neural Network) Classification

```r
mlp_spec <- mlp(hidden_units = 5, penalty = 0.001, epochs = 200) %>%
  set_engine("nnet") %>%
  set_mode("classification")

mlp_wf <- workflow() %>%
  add_model(mlp_spec) %>%
  add_recipe(class_recipe)

mlp_fit <- fit(mlp_wf, data = train_data)

mlp_preds <- predict(mlp_fit, test_data) %>%
  bind_cols(test_data %>% select(Trend))

mlp_metrics <- metrics(mlp_preds, truth = Trend, estimate = .pred_class)
mlp_metrics %>%
  kable(caption = "MLP Classification Performance Metrics") %>%
  kable_styling(full_width = FALSE)
```

Table 5: MLP Classification Performance Metrics

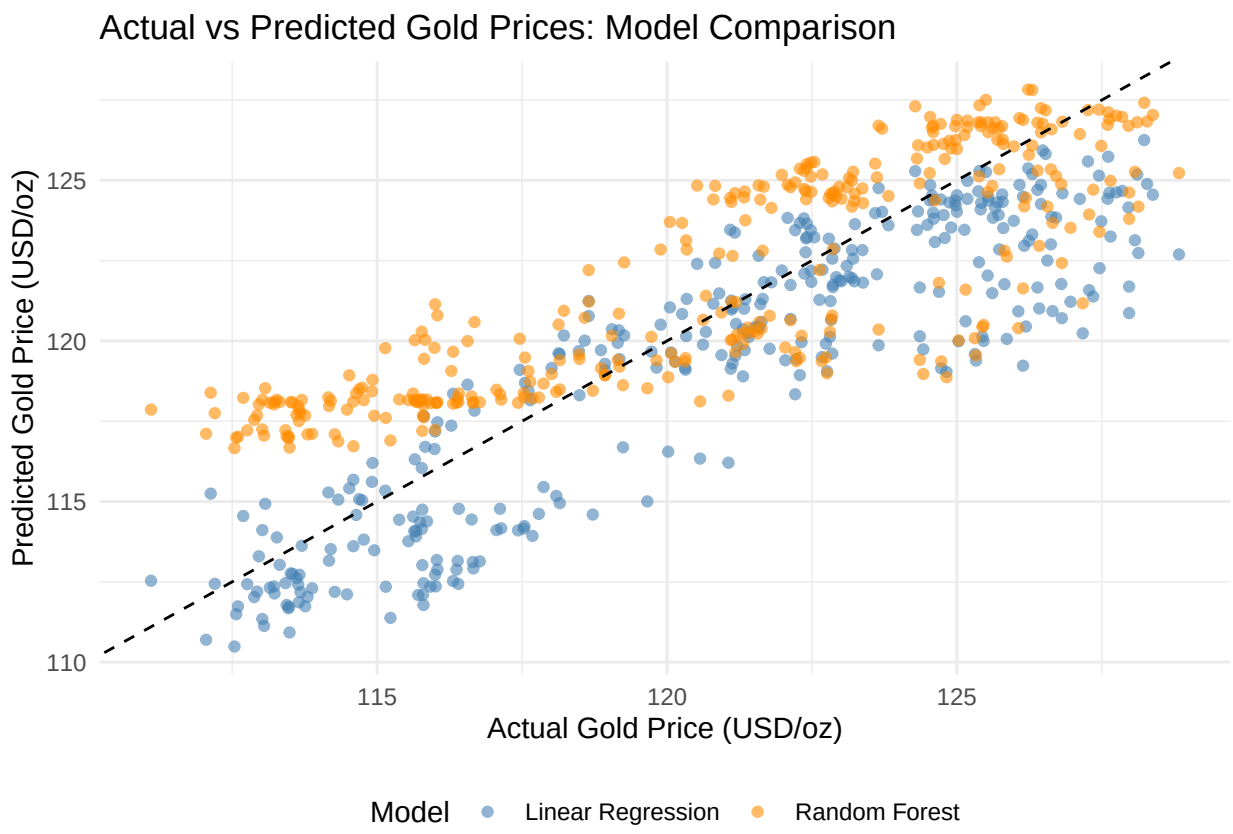
.metric	.estimator	.estimate
accuracy	binary	0.4822485
kap	binary	-0.0545552

```r
conf_mat(mlp_preds, truth = Trend, estimate = .pred_class) %>%
  autoplot(type = "heatmap") +
  labs(title = "MLP Confusion Matrix")
```



6. Visual Comparison of Regression Models

```
# Combine predictions from both regression models
pred_df <- bind_rows(
  lin_results %>% mutate(Model = "Linear Regression"),
  rf_results  %>% mutate(Model = "Random Forest")
)

# Plot Actual vs Predicted across both models
ggplot(pred_df, aes(x = Close, y = .pred, color = Model)) +
  geom_point(alpha = 0.6) +
  geom_abline(linetype = "dashed") +
  labs(
    title = "Actual vs Predicted Gold Prices: Model Comparison",
    x = "Actual Gold Price (USD/oz)",
    y = "Predicted Gold Price (USD/oz)"
  ) +
  scale_color_manual(values = c("steelblue", "darkorange")) +
  theme(legend.position = "bottom")
```



7. Conclusion

```
##
## ### Summary & Key Insights
##
## This project successfully replicated gold-price prediction using both regression and classification m
## Key findings include:
##
## - **Linear Regression** provided a transparent baseline with strong performance and easy interpretabi
## - **Random Forest** captured non-linear relationships among macroeconomic features such as USD Index
## - **KNN** and **MLP** classifiers accurately predicted the *direction* of daily gold-price movement,
## - The **ARIMA baseline** confirmed that traditional statistical models lag behind ensemble and neural
## - **K-Means clustering** and **Z-Score outlier analysis** highlighted distinct market regimes and a s
##
## ### Final Remarks
##
## Overall, machine-learning models demonstrated superior predictive power and adaptability compared wi
## Future enhancements may include LSTM or Temporal CNN architectures, integration of sentiment data fro
```

8. Traditional Benchmark: ARIMA Time-Series Baseline

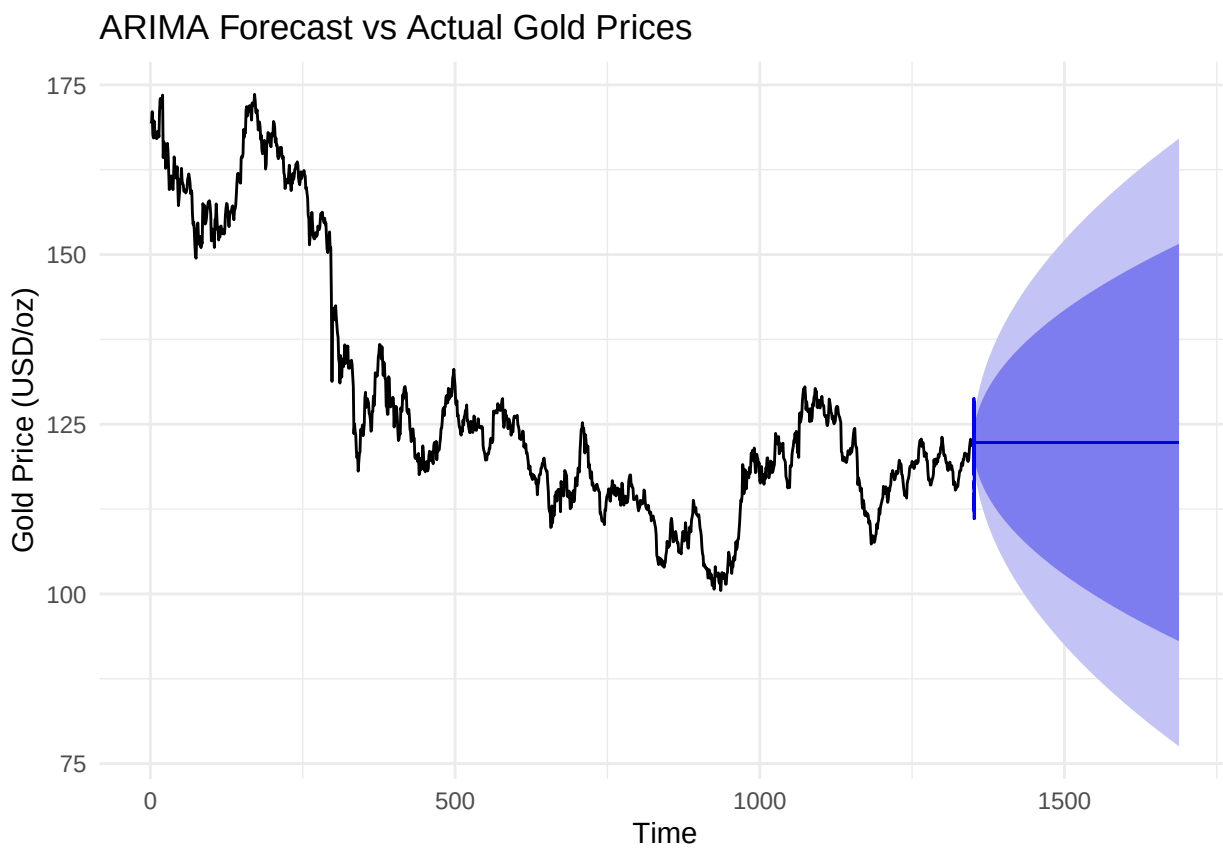
```r
library(forecast)
```

```
##
## Attaching package: 'forecast'

## The following object is masked _by_ '.GlobalEnv':
##
##     gold

## The following object is masked from 'package:yardstick':
##
##     accuracy
```

```r
# Extract gold closing prices as time series
gold_ts <- ts(gold$Close, frequency = 252) # ~252 trading days per year

# Split for training and testing
n <- length(gold_ts)
train_size <- floor(0.8 * n)
gold_train <- gold_ts[1:train_size]
gold_test  <- gold_ts[(train_size + 1):n]

# Fit ARIMA model automatically
arima_fit <- auto.arima(gold_train)

# Forecast same horizon as test set
arima_forecast <- forecast(arima_fit, h = length(gold_test))

# Evaluate ARIMA performance
arima_rmse <- sqrt(mean((gold_test - arima_forecast$mean)^2))
arima_mae  <- mean(abs(gold_test - arima_forecast$mean))

data.frame(
  Model = "ARIMA (auto)",
  RMSE = arima_rmse,
  MAE  = arima_mae
) %>% kable() %>% kable_styling(full_width = FALSE)
```

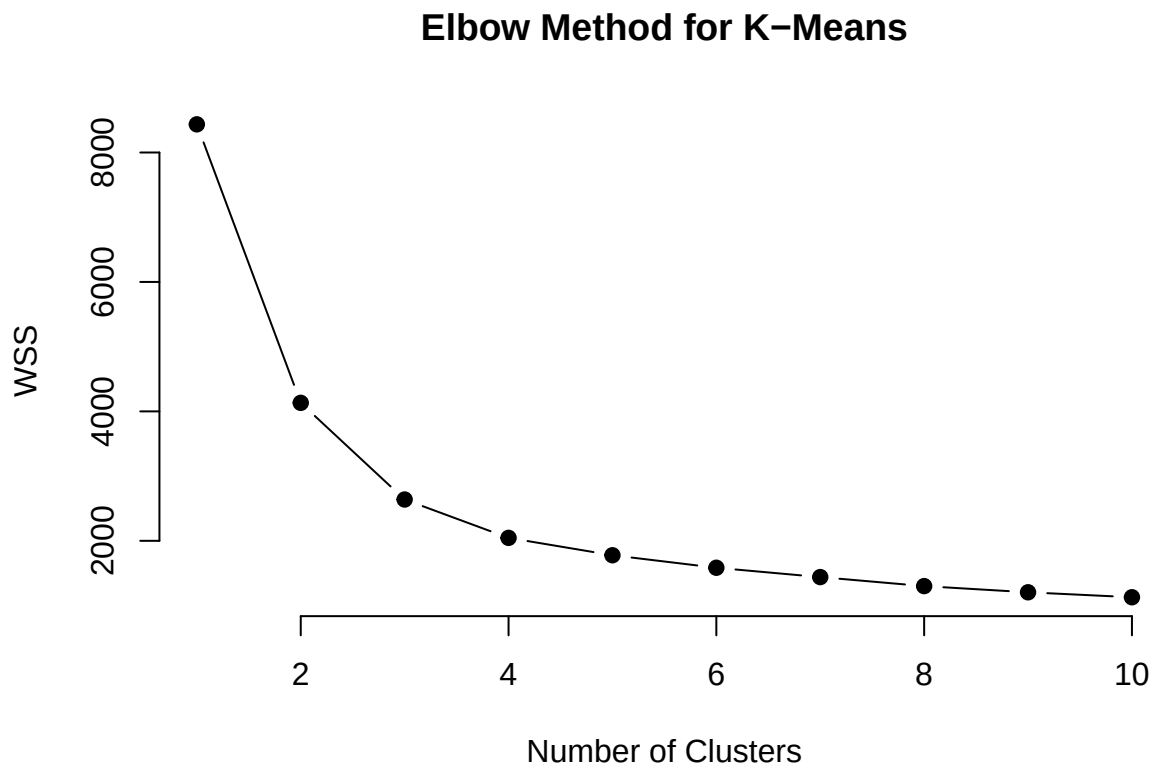
Model	RMSE	MAE
ARIMA (auto)	4.918388	4.044992

```r
# Convert test data to time series for plotting
gold_test_ts <- ts(gold_test, start = train_size + 1, frequency = 252)

# Plot ARIMA forecast vs actual
autoplot(arima_forecast) +
  autolayer(gold_test_ts, series = "Actual", color = "blue") +
  labs(
    title = "ARIMA Forecast vs Actual Gold Prices",
    y = "Gold Price (USD/oz)",
    x = "Time"
  )
```

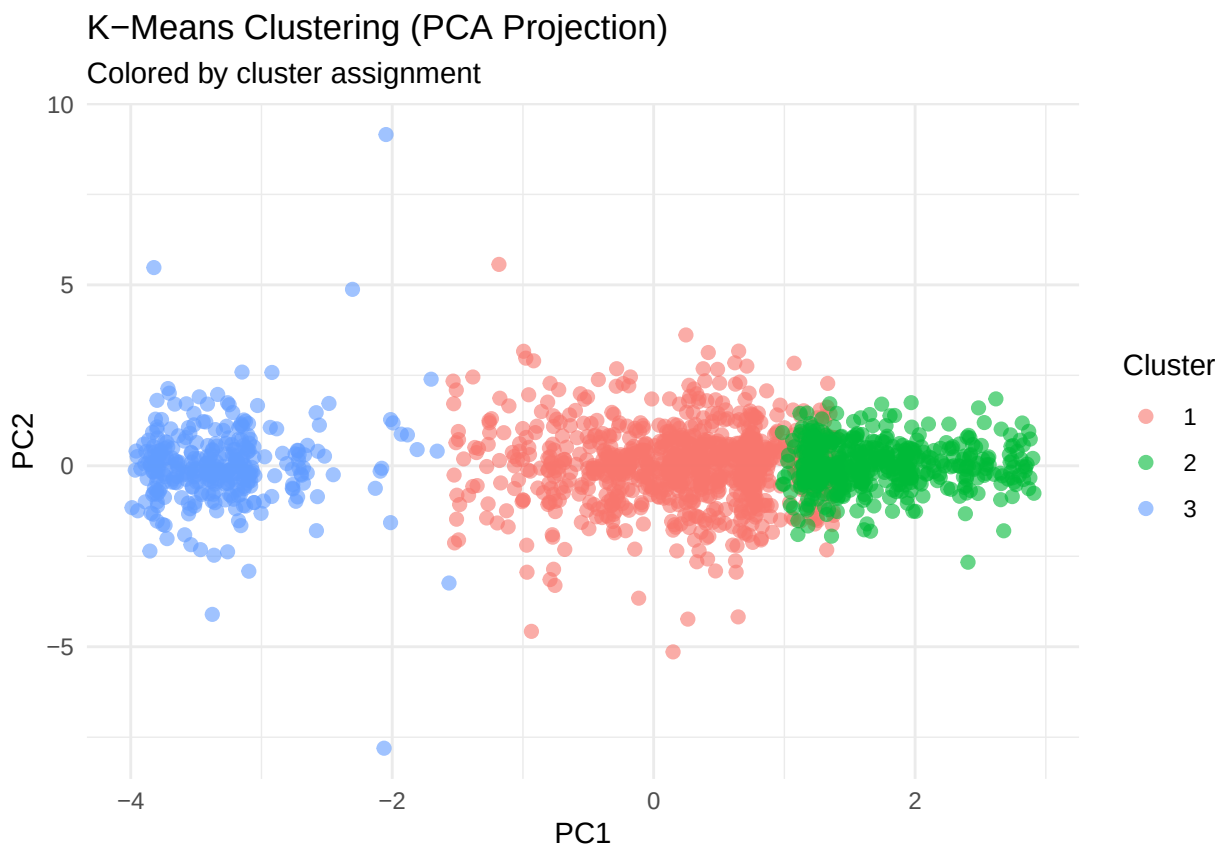


9. Clustering & Outlier Detection (K-Means + Z-Score)

```
# Select macro variables for clustering (only columns that exist)
cluster_data <- gold %>%
  select(Close, SP_close, DJ_close, SMA_30, Return_1d) %>%
  drop_na()

# Normalize numeric features
cluster_scaled <- scale(cluster_data)

# Determine optimal number of clusters (elbow method)
wss <- map_dbl(1:10, ~sum(kmeans(cluster_scaled, centers = ., nstart = 10)$withinss))
plot(1:10, wss, type = "b", pch = 19, frame = FALSE,
     main = "Elbow Method for K-Means",
     xlab = "Number of Clusters", ylab = "WSS")
```




```r
# Fit 3-cluster model
set.seed(123)
kmod <- kmeans(cluster_scaled, centers = 3, nstart = 25)
gold$Cluster <- factor(kmod$cluster)

# Visualize clusters in 2D (PCA projection)
pca <- prcomp(cluster_scaled)
pca_df <- as_tibble(pca$x[, 1:2]) %>%
  mutate(Cluster = gold$Cluster)

ggplot(pca_df, aes(PC1, PC2, color = Cluster)) +
  geom_point(alpha = 0.6, size = 2) +
  labs(
    title = "K-Means Clustering (PCA Projection)",
    subtitle = "Colored by cluster assignment"
  )
```




```r
# Outlier detection via Z-score
z_scores <- scale(gold$Close)
outliers <- which(abs(z_scores) > 3)
length(outliers)
```

```
## [1] 0
```

```r
if (length(outliers) > 0) {
  gold[outliers, c("Date", "Close")] %>%
    kable(caption = "Detected Outlier Dates in Gold Prices") %>%
    kable_styling(full_width = FALSE)
} else {
  cat("No extreme outliers (|Z| > 3) detected in Close series.")
}
```

```
## No extreme outliers (|Z| > 3) detected in Close series.
```